RSE4502:
Robotics Manipulation

JetArm Robot Mini Project 2

Yuan Qilong

Senior Professional Officer

Learning Objective

- Learning Robot Manipulation Programming with Mini Projects
- Learning Robot Programming for Pick and Placing
- Learning Robot Motion and Trajectory Planning and Obstacle Avoidance
- Learn Object Tracking with AprilTag
- Practice Coordinate Frame Pose Representation and Trasformations (tf, pose, etc.)
- Learn to Apply Robot Forward / Inverse Kinematics

SINGAPORE INSTITUTE OF TECHNOLOGY

This slides is in association with:
https://ql-yuan.github.io/Jetarm_Session2_Instruction/

AprilTag Pose Detection

Object Tag Pose Detection

- Attach tag on the long blocks (For example: tag 2 for blue, tag 3 for red)
- Track tag pose with tag_tracking_basic.py
- Validate pose correctness with ik_english.py
- Figure out the poses of the object tags

Use camera dt_apriltag library to detect apriltag pose

- Node file: tag_tracking_basic.py
  - The camera frame: rgbd_cam_color_optical_frame
  - It outputs the rotation and position of each tag in camera frame:
    - Check next page
  - User need to figure out the tag pose in robot base_link for IK calculation
    - User can calculate yourself, and check whether the value I collected is correct (**in next slide**)
    - Test the recorded tag pose with ik_english.py (set x,y, z, and pitch angle)
    - The measurement from dt_apriltag is not accurate if camera is not well calibrated.
    - Please calibrate you camera
    - User can try to fine-tune the object pose value for correct grasping. (ik_english.py can be used for validation)

I am tag2

```
[[ 0.99230236  0.12324839 -0.01207755]
 [-0.10[illegible]  0.87[illegible]  0.46638726]
 [ 0.06815022 -0.46204451  0.88423437]]
```
Rotation in Camera Frame

```
[[-0.03081049]
 [-0.02[illegible]]
 [ 0.21300428]]
```
Position in Camera Frame

I am tag3

```
[[ 0.02172595  0.99973692  0.00735366]
 [-0.88446163  0.01579058  0.46634567]
 [ 0.46610686 -0.01663583  0.88457201]]
[[ 0.05522355]
 [-0.03148454]
 [ 0.2170428 ]]
```

Camera model: fx = 452.533, fy = 452.533, cx = 325.613, cy = 240.352 rgbd_cam

ROS Node Tag Tracking in Python

- Python File
  - Copy file tag_tracking_basic.py to your Jetson Orin Nano, ~/Desktop for example
- For Robot Control
  - Firstly, start a terminal window and run:
    - exec bash
    - sudo systemctl stop jetarm_bringup.service
    - roslaunch jetarm_bringup base.launch
  - To run this IK node, start a terminal window and run:
    - exec bash
    - cd ~/Desktop
    - python3 tag_tracking_basic.py

Explanation of tag_tracking_basic.py(1)

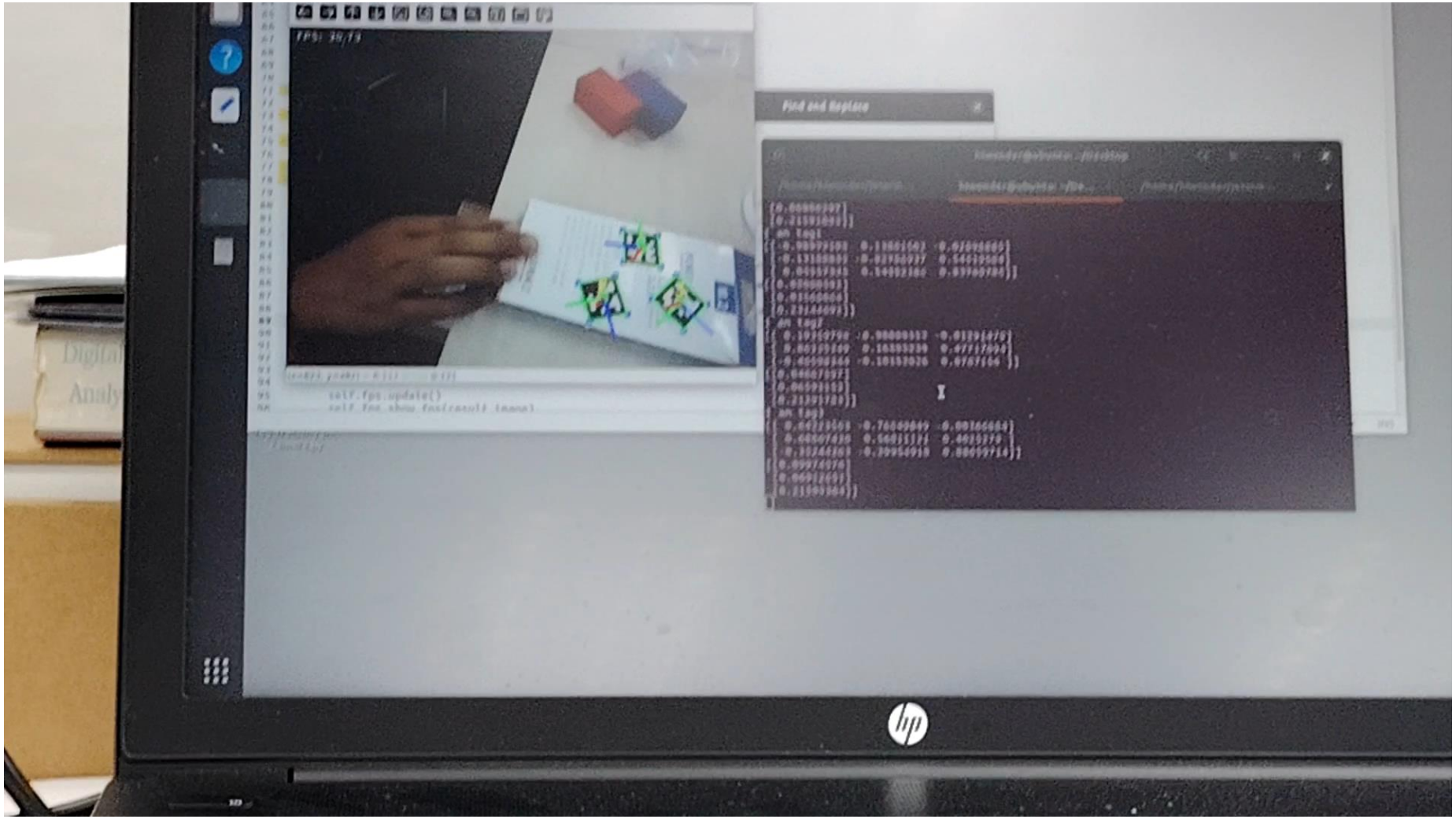
SINGAPORE INSTITUTE OF TECHNOLOGY

```
tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY), True,
[452.533, 452.533, 325.613, 240.352], 0.025)
#True means tracking pose of tags
#camera model: [fx,fy,cx,cy] is [452.533, 452.533, 325.613, 240.352].

#You should replace this value based on your camera calibration data.

camera_matrix = np.array([[452.533, 0, 325.613],
                          [0, 452.533, 240.352],
                          [0, 0, 1]], dtype=np.float32)

dist_coeffs = np.zeros((4, 1))  # Assuming no distortion
tag_size = 0.025  # in meters
```

Explanation of tag_tracking_basic.py(2)

```
R0=tag.pose_R
P0=tag.pose_t
corners = tag.corners
# Draw the detected tag's corners
result_image = cv2.polylines(result_image, [np.int32(corners)], isClosed=True, color=(0, 255, 0), thickness=2)
# Draw the coordinate frame (axes) on the tag
cv2.drawFrameAxes(result_image, camera_matrix, dist_coeffs, R0, P0, 0.025)  # 0.025 is the length of the axes in meters
```

The tracking results



Camera Calibration

ROS Node For Calibration Image Collection

- Python File
  - Copy file image_collector.py to your Jetson Orin Nano, ~/Desktop for example
- For Robot Control
  - Firstly, start a terminal window and run:
    - exec bash
    - sudo systemctl stop jetarm_bringup.service
    - roslaunch jetarm_bringup base.launch
  - To run this node, start a terminal window and run:
    - exec bash
    - cd ~/Desktop
    - python3 image_collector.py
- To Use the app for image collection:
  - Pose your checkerboard for image collection.
  - Simply **Left click your mouse** to save the image (image name will start from 0.png, 1.png,…)
  - Collect as enough image as you need, 10 for example.

Frame Descriptions and Kinematics

Transformations to Know for Calculations

Pose of camera frame in link5 frame:

${}^{link5}_{camera}T$= translation([-0.045,0,0.02]) *rotz(-90deg)

$$ {}^{link5}_{camera}T \cong \begin{bmatrix} 0 & 1 & 0 & -0.045 \\ -1 & 0 & 0 & 0 \\ 0.0 & 0 & 1 & 0.02 \\ 0.0 & 0 & 0 & 1 \end{bmatrix} $$

x,y,z= [-0.045,0,0.02]

yaw,pitch,roll=-1.57,0,-0.0

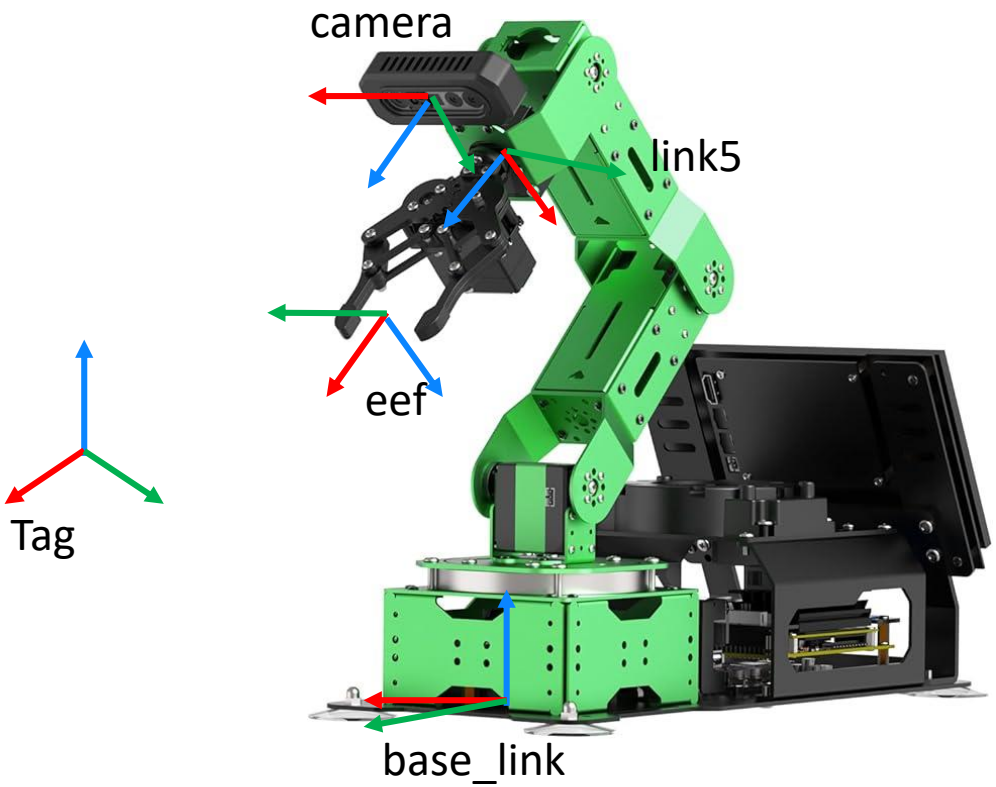
Pose of eef frame in link5 frame:

${}^{link5}_{eef}T$= translation([0.0,0.0, 0.11054687369]) *roty(-90deg)

x,y,z= [0.0,0.0, 0.11054687369]

yaw,pitch,roll=0.0,-1.57,0.0

Attention !



<node pkg="tf" type="static_transform_publisher" name="eef_to_camera_transform" args="-0.045 0.0 0.02 -1.57 0 0 link5 rgbd_cam_color_optical_frame 100"/>

x, y, z, yaw, pitch, roll

Forward Kinematics: eef pose in base_link

SINGAPORE INSTITUTE OF TECHNOLOGY

```
fk = ForwardKinematics(debug=True)
servo_list =[500,550,285,150,500]
print(servo_list)
res = fk.get_fk(pulse)  #get forward kinematics
print('coordinate:  ', res[0])
print('Quaternion:  ', res[1])
print('Conert Quaternion to rpy:  ', transform.qua2rpy(res[1]))
```

How to transform between servo_list and joint angles:

```
servo_list = transform.angle2pulse([[0.0, -0.20943950000000003, -0.9005898500000001, -1.4660765000000002, 0.0]])[0]
servo_list=[math.floor(float(x)) for x in servo_list]

angle = transform.pulse2angle(servo_list)  # in rad
```

Access to eef poses and robot joint angles

- Use service to get pose of eef in base_link
- rosservice call **/kinematics/get_current_pose**

```
pose:
  position:
    x: 0.34001989226271
    y: -0.01282452402598488 1
    z: 0.1781471829141277 6
  orientation:
    x: 7.89519287736612e-05
    y: 0.004188033828972482
    z: -0.018848274358633444
    w: 0.9998135809704434
```

- This service can replace forward kinematics

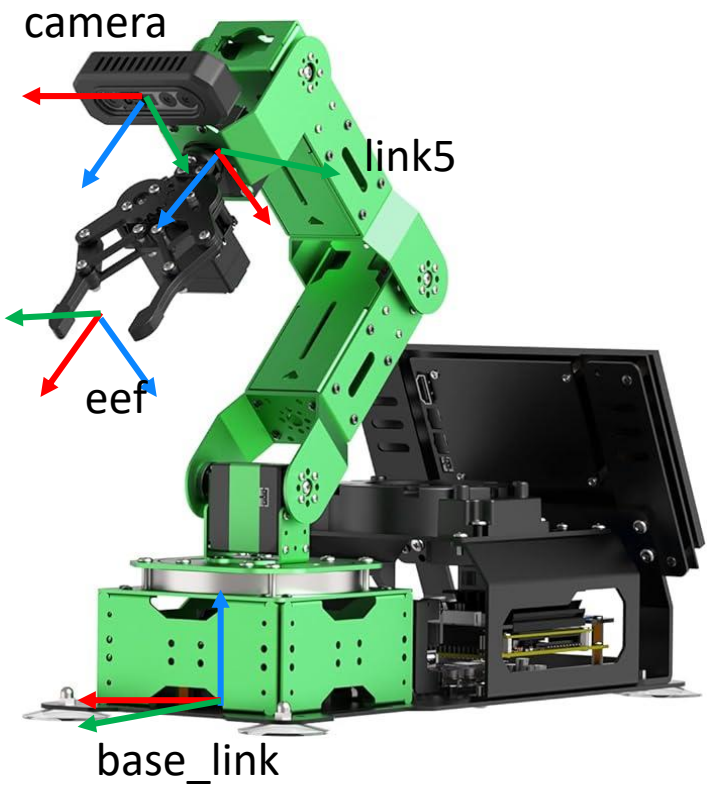
Get joint angles from topic joint_states

$rostopic echo joint_states

```
---
header:
  seq: 50849
  stamp:
    secs: 1175
    nsecs: 147607743
  frame_id: ''
name:
  - joint1
  - joint2
  - joint3
  - joint4
  - joint5
  - r_joint
position: [0.0, 0.20943950000000003, -0.9005898500000001, -1.4660765000000002, 0
.0, 1.2566370000000002]
velocity: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
effort: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
---
```

Nodes on Inverse Kinematics

- IK reference frame: base_link; End effector frame:eef
- Check the file “ik_english.py” we share before



Robot Control Commands

Direct approach to command robot servo values:

SINGAPORE INSTITUTE OF TECHNOLOGY

Controlling robot with bus_servo_control.set_servos

```
self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur, queue_size=1)
bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 450), (3, 285), (4, 150), (5, 500), (10, 200)))
```

Publishing TF and Visualization

Briefing on tf publication for frame visualization

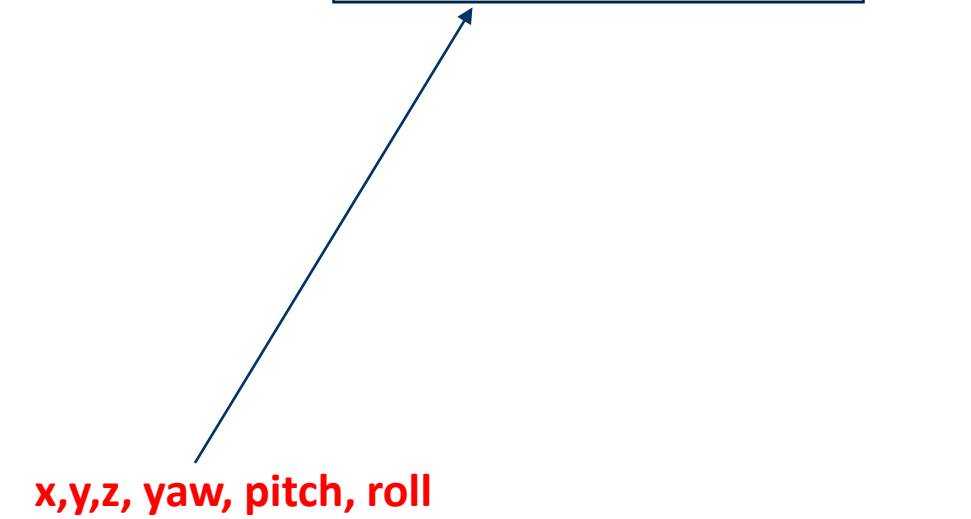
Only for your consideration. You may use different method.

- Use camera dt_apriltag library to detect apriltag pose
  - Publish tag pose in camera
  - Publish tf pose: camera in link5
  - Figure out pose of tag in base_link of JetArm
- Define Initial and Goal pose of objects
- Inverse Kinematics
- Motion Planning for exercises

Eye in Hand Pose: camera pose in link5

- tf transform for rgbd_cam_color_optical_frame in link5

```
<?xml version="1.0" encoding="UTF-8"?>
<launch>
  <node pkg="tf" type="static_transform_publisher" name="eef_to_camera_transform" args="-0.045 0.0 0.02 -1.57 0 -0.1 link5
rgbd_cam_color_optical_frame 100"/>

</launch>
```



To visualize frames in rviz

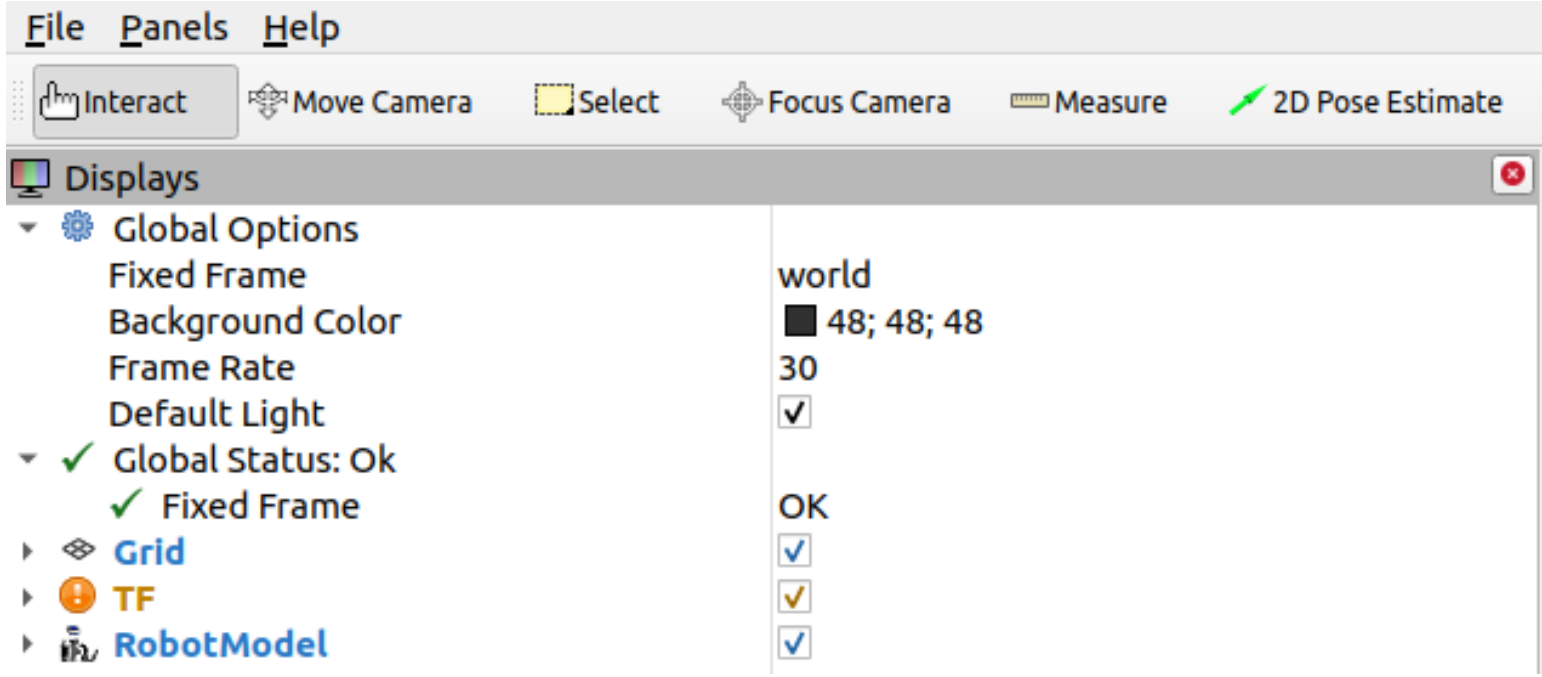
- Launch base.launch in jetarm_bringup package
  - Firstly, start a terminal window and run:
    - exec bash
    - sudo systemctl stop jetarm_bringup.service
    - roslaunch jetarm_bringup base.launch
- Download and put tf_hand2camera.launch to jetarm_bringup/launch folder
- Launch tf_hand2camera.launch

- To track and publish tf of tags in camera frame:
  - Download tag_tracking_basic_tf.py and execute it.
  - Run rviz in a new terminal and setup rviz following next slides.

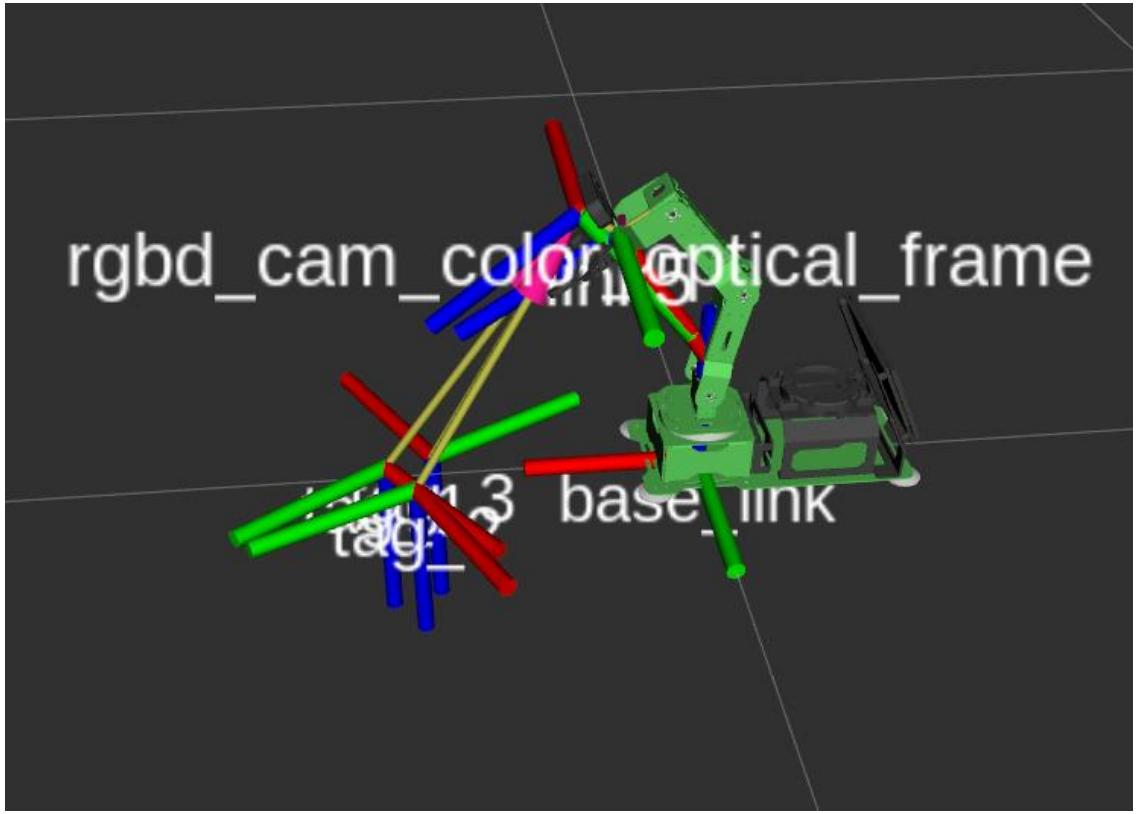
- Confirm your tag is tracked as expected through visualization
- To validate your calculation, you can publish TF to check
  - For example, publish tf to check if your tag pose calculation in base_link is correct.

Visualization in RVIZ

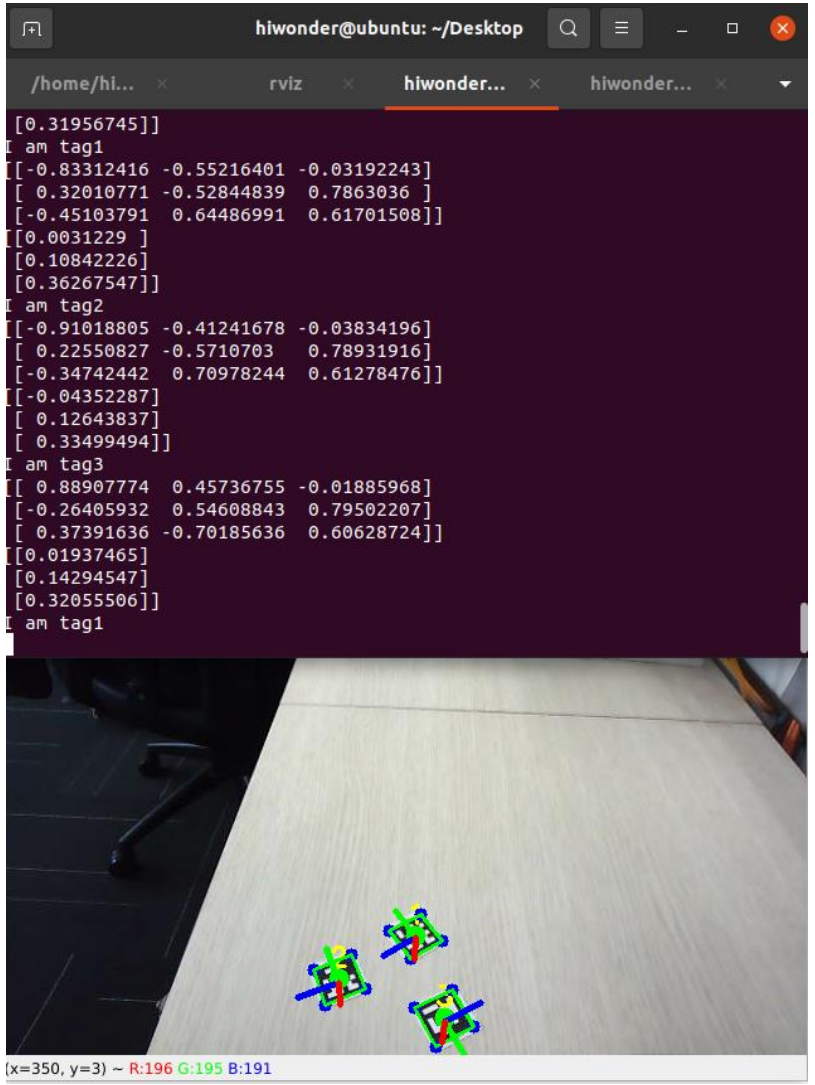
- Change Fixed Frame to **world**
- Add In Following Items:
- TF Display:
  - base_link
  - link5
  - rgbd_cam_color_optical_frame
  - tag1,…
- RobotModel



Publishing Tag tf



What you see in: rviz



Pick and Place

◆ Robot Arm Pick-and-Place Flow Diagram

SINGAPORE INSTITUTE OF TECHNOLOGY

- Start
- ↓
- Camera Intrinsic Calibration (once)
- ↓
- Eye-in-Hand Calibration (once, not applied here)
- ↓
- Capture Image
- ↓
- Detect Object
- ↓
- Estimate Object Pose
- ↓
- Transform Pose to Robot Base Frame
- ↓
- Generate PICK Waypoints:
- • Approaching Pose
- • Grasping Pose
- • Leaving Pose
- ↓
- Solve IK (Pick Waypoints)
- ↓
- Plan Joint Trajectory
- ↓
- Safety Check
- ↓
- Execute PICK:
- → Move to Approaching
- → Move to Grasping
- → Close Gripper
- → Move to Leaving
- ↓
- Generate PLACE Waypoints:
- • Approaching Pose
- • Placing Pose
- • Leaving Pose
- ↓
- Solve IK (Place Waypoints)
- ↓
- Plan Joint Trajectory
- ↓
- Safety Check
- ↓
- Execute PLACE:
- → Move to Approaching
- → Move to Placing
- → Open Gripper
- → Move to Leaving
- ↓
- Repeat

SINGAPORE INSTITUTE OF TECHNOLOGY

End